Candy Mountain - Sprint Plan

Project: Candy Mountain (zero-UI co-op space-portal site) **Repository:** github.com/melonmelonz/candy-mountain **Prepared:** 2026-06-05 **Author:** Penn Porterfield

---

Product Summary

Candy Mountain is a zero-UI, fully diegetic co-op web experience. Players (called "drifters") share a single arena split by a central seam. The crowd distributes itself across the two halves to charge a portal; when the portal opens it sends everyone to a daily curated destination link. There is no HUD, menu, or button: every piece of feedback is the world itself reacting.

Tech stack: Preact + Vite + TypeScript + Bun, Canvas 2D rendering. Cloudflare Pages (static SPA) plus a sibling Cloudflare Worker hosting the `PortalRoom` Durable Object (SQLite-backed) for authoritative shared state.

Sprint 1 - Engine and Multiplayer Foundation

Dates: 2026-06-04 (1 day) **Sprint Goal:** Stand up a playable, networked arena: local movement with client-side prediction, authoritative server state over WebSockets, and the core charge-the-portal loop with a diegetic open-and-redirect sequence.

Committed Backlog

ID	User Story	Points	Status
CM-1	As a player, I can move my drifter with WASD/arrows so I can explore the arena.	2	Done
CM-2	As a player, my movement feels responsive even under network latency (client-side prediction + remote interpolation).	3	Done
CM-3	As a player, I see other players move in real time so the space feels shared.	3	Done
CM-4	As the system, I maintain authoritative room state in a Durable Object and broadcast it to all clients.	5	Done
CM-5	As a player, I see the portal charge as the crowd covers the required spots, with no HUD.	5	Done
CM-6	As a player, when the portal opens I am carried through a cinematic redirect to the daily link.	3	Done
CM-7	As a player, I get ambient audio, portal transmissions, and hidden easter eggs that reward curiosity.	2	Done

Committed: 23 points. **Completed:** 23 points.

Definition of Done

- Feature implemented behind the diegetic (no-UI) constraint.
- `bunx tsc --noEmit` passes for both client and worker.
- `bun test` green.
- Deployed to Cloudflare Pages + Worker and smoke-tested live.

Sprint 1 Outcome

All committed stories shipped. The networked loop is fully playable: input tracker (CM-1), client world model with self-prediction and remote interpolation (CM-2, CM-3), WebSocket net client with a Pages `/room` proxy and the Durable Object reducer/tick (CM-4), charge-driven portal visualization with covered-pad pulses (CM-5), open-burst redirect transition with single-fire navigation guard (CM-6), and ambience + transmissions + easter eggs (CM-7).

Sprint 2 - Art Overhaul, Inverted Far Side, and Polish

Dates: 2026-06-05 (1 day) **Sprint Goal:** Elevate the prototype to demo quality: replace placeholder art with a manifest-driven sprite roster and animated portals, add the inverted far-side mechanic for depth, give the camera cinematic life, and complete a performance and security pass.

Committed Backlog

ID	User Story	Points	Status
CM-8	As a player, I control one of many distinct 8-direction pixel-art characters instead of a single placeholder sprite.	8	Done
CM-9	As a player, the portal is a hand-crafted animated warpgate, and the destination rotates per solve cycle rather than per calendar day.	5	Done
CM-10	As a player, crossing the central seam inverts my controls, hinted once by a whisper and marked by a photo-negative shimmer, so the far side feels like another realm.	5	Done
CM-11	As a player, the camera frames me on spawn, hides the portal until I find it, and dollies out cinematically when the ritual completes.	5	Done
CM-12	As a player, a high-fidelity parallax space background (real NASA planet photos, crisp stars) sits behind the retro foreground and shifts as I move.	5	Done
CM-13	As the system, portal openings are persisted to Durable Object SQLite storage for a durable history.	2	Done
CM-14	As a player, the experience loads fast and is safe against malformed/malicious client input (performance + security pass).	3	Done

Committed: 33 points. **Completed:** 33 points.

Definition of Done

Same as Sprint 1, plus:

- Visual changes verified by screenshot before deploy.
- No client-supplied value trusted without server-side validation.

Sprint 2 Outcome

All committed stories shipped.

- **CM-8:** Manifest-driven renderer for a 16-character PixelLab roster stored under `public/sprites/`; characters render in 8 directions with idle / walk / rotation-still fallback. Facing widened from 4 cardinals to the full 8 atlas directions so diagonal movement shows diagonal poses.
- **CM-9:** Six daily-rotating animated warpgates replace the procedural portal; destination resolves by per-solve `cycleIndex` (derived from the `portal_opens` count) with the server broadcasting the active `gateId`.
- **CM-10:** Inverted controls past the seam, a one-time first-crossing whisper, and a subtle photo-negative shimmer on any far-side drifter.

- **CM-11:** Spawn framing that keeps the portal off-screen until discovered, with a cinematic dolly-out lock when the ritual completes; fixed a bug where the camera latched onto the portal before the real spawn arrived.
- **CM-12:** High-def parallax background with three star layers, drifting nebulae, and three real NASA planet photos (Jupiter, Earth, Mars); stars sharpened; camera-coupled parallax so the cosmos shifts with movement.
- **CM-13:** `portal_opens` SQLite table (`day_id`, `opened_at`, `present`), written on each open. First production use of the DO SQLite storage.
- **CM-14:** Performance: downscaled planet images (~1.5 MB to ~83 KB), preload alongside sprite atlases to kill first-frame pop-in, and cache the base background gradient instead of rebuilding it every frame. Security: confirmed server-side validation of all client input (IDs server-assigned via `crypto.randomUUID`, movement finite-checked and clamped to arena bounds, facing whitelisted, cosmetics sanitized, chat rate-limited and length-capped). Also bumped walk speed and parallax strength for stronger motion feedback.

Cross-Sprint Definition of Done (reference)

A story is Done when:

1. Code is implemented and merged to `main`.
2. Type checks pass: `bunx tsc --noEmit` (client) and the same in `portal-room/`.
3. Automated tests pass: `bun test`.
4. The change is deployed (`bun run deploy:worker` and/or `bun run deploy:pages`).
5. The diegetic constraint is honored: no HUD, menus, or buttons.
6. For visual work, the result is screenshot-verified before deploy.

Risks and Constraints

- **ASCII-only commit messages.** The wrangler deploy GitHub Action breaks on unicode, so all commit messages stay ASCII (use `->` not arrows).
- **Pages + Durable Objects topology.** Pages cannot declare a DO inline; the DO lives in a sibling Worker bound by `script_name`. Link/spawn data is bundled into the Worker, so link changes require a Worker redeploy.
- **Two remotes.** `origin` (melonmelonz) is the push target; `partner` (summer-marie) must never be pushed to. Verify before every push.

Backlog (Not Committed - Future Sprints)

- Surface the `portal_opens` log diegetically (e.g. "opened N times today") or expose a read endpoint; the data is written but nothing reads it yet.
- Persist chat history across DO eviction (currently in-memory, last 10).
- Server-side cosmetics persistence (currently client `localStorage` only).
- Tune inversion difficulty if playtests show the portal rarely opens (knob: `chargeDecayPerTick` in `src/game/config.ts`).

Retrospective Notes

What went well: Both sprints delivered 100% of committed points. The diegetic-first discipline held across every feature. Catching the planet pop-in and the spawn-camera bug before they shipped to players.

What to watch: Story sizing skewed large in Sprint 2 (several 5- and 8-pointers in a one-day window); future sprints should slice art-heavy stories into thinner vertical increments. Visual regressions are easy to miss without screenshots, so screenshot-before-deploy is now part of the DoD.